[20261008] INFOFP - Functioneel programmeren - 1 - USP

Cursus: BETA-INFOFP Functioneel programmeren (INFOFP)

Tijdsduur: 2 uur

Aantal vragen: 4

[20261008] INFOFP - Functioneel programmeren - 1 - USP

Cursus: Functioneel programmeren (INFOFP)

Aantal vragen: 4

1 In this question we consider the following type synonym that we use to represent weighted collections of values

```
type Weighted a = [(a, Int)]
```

An entry (x,w) gives a value x and its weight w. All weights are non-negative integers. Values may repeat, and may have zero weight. Furthermore, the empty list is allowed. The total weight of a value x is the sum of the weights of all entries where the first element is x.

a. [6pt] Complete extractWeight. Return the sum of all weights attached to the value x, together with the input list with all entries for x removed. Preserve the order and weights of all other entries.

Fill a1 with a list comprehension and a2 with a predicate for filter. Do not use explicit recursion.

```
extractWeight :: Eq a => a -> Weighted a -> (Int, Weighted a)
extractWeight x xs = (sum a1 , filter ( a2 ) xs) where
  a1 = a.  ..........(3 pt.)
  a2 = b.  ..........(3 pt.)
```

For example:

```
extractWeight 'a' [('a',2),('b',0),('a',3),('c',4)] == (5,
[('b',0),('c',4)])
extractWeight 'z' [('a',2)] == (0, [('a',2)])
extractWeight 'a' [] == (0, [])
```

b. [6pt] Imagine placing tickets (the values) in a row. An entry (x,w) contributes w consecutive tickets labelled x, in the same order as the entries in the list. Number all tickets starting at zero. A zero-weight entry contributes no tickets.

For example, for [('a',2),('b',0),('c',3)] we have:

```
ticket number:   0   1   2   3   4
ticket label:   'a' 'a' 'c' 'c' 'c'
```

Using direct recursion, implement:

```
ticket :: Int -> Weighted a -> Maybe a
```

So that 'ticket i ws' returns Just x when the ticket with ticketnumber i exists and is labelled x. Return Nothing for a negative number,
a number beyond the last ticket, or an input with no tickets. Repeated labels are allowed and are not merged.

Your implementation of 'ticket i ws' should run in time linear in the length of 'ws' (which may be much smaller than the number i).

For example:

```
ticket 2 [('a',2),('b',0),('c',3)] == Just 'c'
ticket 5 [('a',2),('b',0),('c',3)] == Nothing
```

```
ticket (-1) [('a',2)] == Nothing
ticket 0 [('a',0)] == Nothing
ticket 0 ([] :: Weighted Char) == Nothing
```

6 pt. **c.**

c. [5pt] Using foldr, implement:

```
uncompress :: Weighted a -> [a]
```

Replace each (x,w) by w copies of x, preserving the order of the entries. You may use replicate and (++), but no explicit recursion or comprehensions. Recall that replicate n x produces a list of n copies of x.
For example:

```
uncompress [('a',2),('b',0),('c',3)] == "aaccc"
uncompress [('a',1),('b',1),('a',1)] == "aba"
uncompress ([] :: Weighted Char) == []
```

5 pt. **d.**

d. [4pt] Write a function 'andThen f ws' that for every entry (x,w) in the input 'ws' replaces it by the entries of f x while multiplying each of their weights
by w. Preserve input order and the order within each f x. Do not merge equal results or discard zero-weight entries.
Complete the code below.

```
andThen :: (a -> Weighted b) -> Weighted a -> Weighted b
andThen _ [] = e. .....(1 pt.)
andThen f ((x,w):xs) =
  let ys = f x
      rest = andThen f. .....(1 pt.)
      in map ( g. .....(2 pt.) ) ys ++ rest
```

For example:

```
andThen (\x -> [(x,1),(x+10,2)]) [(3,2),(4,0)] ==
[(3,2),(13,4),(4,0),(14,0)]
andThen (const ([] :: Weighted Bool)) [('a',3)] == []
andThen (\_ -> [('a',1)]) [(True,2),(False,3)] == [('a',2),('a',3)]
```

e. [5pt] The following helper is supplied. It removes zero-weight entries and combines equal results by adding
their weights. Results are ordered by their first non-zero-weight occurrence.

```
coalesce :: Eq a => Weighted a -> Weighted a
coalesce = foldr step []
  where
```

```
  step (x,w) acc
    | w == 0 = acc
    | otherwise =
        let (v,rest) = extractWeight x acc
        in (x,w+v) : rest
```

For example:

```
coalesce [('a',0),('b',2),('a',3),('b',1)] == [('b',3),('a',3)]
coalesce [('a',0),('b',0)] == []
```

We use a wrapper:

```
data WeightedN a = W (Weighted a) deriving Show
```

Implement an Eq instance with the weakest sufficient type-class constraint. W xs and W ys are equal exactly when their coalesced lists are equal. In this question, the order in those lists matters; do not sort them.
For example:

```
> W [('a',2),('b',0),('a',3)] == W [('a',5)]
True
> W [('a',1),('b',1)] == W [('b',1),('a',1)]
False
> W [('a',0)] == W ([] :: Weighted Char)
True
```

5 pt. **h.**

2 In this question, we ask you to write some code that uses a novel datatype. The datatype of so-called "traces" represents a simplified version of representations commonly used in popular frameworks for computing derivatives. You do not need to know anything about derivatives for this question.

A trace records a computation using multiplication, together with the value computed at each node:

```
data Trace a = Input a Float
             | Constant Float
             | Multiply Float (Trace a) (Trace a)
                deriving Show
```

Input x v records an input occurrence labelled x with current value v. Repeated labels are allowed. Multiply n l r stores the product of its children's values in n. All supplied traces satisfy this invariant.

This helper is supplied; it reads the root's value without visiting children.

```
value :: Trace a -> Float
value (Input _ v) = v
value (Constant c) = c
value (Multiply n _ _) = n
```

For example, this records (x * 3) * x with x = 2:

```
demo :: Trace Char
demo = Multiply 12 (Multiply 6 (Input 'x' 2) (Constant 3)) (Input 'x' 2)
```

a. [4pt] Using direct recursion, implement:

```
inputs :: Trace a -> [a]
```

List the input labels in left-to-right order, preserving repeated labels. Constants contribute nothing. There is no efficiency requirement.
For example:

```
inputs demo == "xx"
inputs (Multiply 6 (Input 'x' 2) (Input 'y' 3)) == "xy"
inputs (Constant 7 :: Trace Char) == []
```

4 pt. **a.**

b. [3pt] Implement the function

```
mulTrace :: Trace a -> Trace a -> Trace a
```

to multiply two traces. If either input is exactly Constant 1, return the other trace unchanged. Otherwise construct one Multiply node with the correct
cached value and the original children in order. Do not perform any other simplification. Your implementation should run in constant time. You may use the helper function 'value' provided above.

Note that 'Input _ 1', or a compound trace whose cached value is 1, is not the same as a 'Constant 1' as the input values may change later, so it must not be removed.
For example:

```
> mulTrace (Constant 1) (Input 'x' 2)
Input 'x' 2.0
> mulTrace (Input 'x' 2) (Constant 1)
Input 'x' 2.0
> mulTrace (Input 'x' 1) (Constant 3)
Multiply 3.0 (Input 'x' 1.0) (Constant 3.0)
> mulTrace (Constant 2) (Constant 3) :: Trace Char
Multiply 6.0 (Constant 2.0) (Constant 3.0)
```

3 pt. **b.**

c. [5pt] Complete the implementation of the function replay below. The function 'replay env t' replaces every input's stored value by env applied to its label, and updates the
caches to preserve the invariant. Preserve all labels, constants and the exact tree shape. Do not simplify the tree.

```
replay :: (a -> Float) -> Trace a -> Trace a
replay env (Input x _) = c. .....(1 pt.)
replay _ (Constant c) = Constant c
replay env (Multiply _ l r) =
  let l' = d. .....(1 pt.)
      r' = e. .....(1 pt.)
      in Multiply ( f. .....(2 pt.) ) l' r'
```

For example:

```
> replay (const 4) demo
Multiply 48.0 (Multiply 12.0 (Input 'x' 4.0) (Constant 3.0)) (Input 'x'
4.0)
> replay (\x -> if x == 'x' then 4 else 5) (Multiply 6 (Input 'x' 2) (Input
'y' 3))
Multiply 20.0 (Input 'x' 4.0) (Input 'y' 5.0)
> replay (const 4) (Multiply 2 (Constant 1) (Input 'x' 2))
Multiply 4.0 (Constant 1.0) (Input 'x' 4.0)
```

d. [6pt] Complete the function inputs' below so that it has the same behaviour as inputs, but takes linear time, even for
unbalanced traces. The helper satisfies the invariant:

```
go :: [a] -> Trace a -> [a]
go acc t = inputs t ++ acc
```

Complete the implementation below. Do not implement the helper by calling inputs or (++). Do not use reverse,
comprehensions or higher-order functions either. Visit each node once and use (:) to build the list.

```
inputs' :: Trace a -> [a]
inputs' t = go g. .....(1 pt.) t
  where
  go acc (Input x _) = h. .....(1 pt.)
```

```
  go acc (Constant _) = i.  ..................................... (1 pt.)
  go acc (Multiply _ l r) = j.  ................................. (3 pt.)
```

For example:

```
inputs' demo == "xx"
inputs' (Constant 7 :: Trace Char) == []
```

Inside the helper:

```
go "z" (Multiply 6 (Input 'y' 2) (Input 'x' 3)) == "yxz"
```

The accumulator belongs after the labels found in the trace.

e. [8pt] A backward pass sends a Float sensitivity down the trace. You only need these rules; you do not need
to derive them.

Node reached with sensitivity s	**What to do**
Constant _	Record no contribution.
Input x _	Record the contribution (x,s).
Multiply _ l r	Send s * value r to l, and s * value l to r.

We implement this in two stages. First the function gradient' replaces every input label x by (x,s), where s is the
sensitivity reaching that occurrence. Preserve the tree shape, input values, constants and caches. Then the function gradient runs gradient' with initial sensitivity 1 and collects the annotated labels in left-to-right
input order. Keep repeated labels and zero contributions. Use inputs' defined above for the collection stage.

Complete the code below. Use value to read the caches; do not recompute subtrees.

```
gradient' :: Float -> Trace a -> Trace (a, Float)
gradient' _ (Constant c) = k.  ..................................... (1 pt.)
gradient' s (Input x v) = l.  ..................................... (2 pt.)
gradient' s (Multiply n l r) =
  Multiply n (gradient' ( m.  ..............................  (1,5 pt.)  ) l)
             (gradient' ( n.  ..............................  (1,5 pt.)  ) r)

gradient :: Trace a -> [(a,Float)]
gradient t = o.  ................................................ (2 pt.)
```

For example, with incoming sensitivity 2, gradient' annotates labels but keeps the original cached product and input values:

```
> gradient' 2 (Multiply 6 (Input 'x' 2) (Input 'y' 3))
Multiply 6.0 (Input ('x',6.0) 2.0) (Input ('y',4.0) 3.0)
```

Examples of the final list returned by gradient:

```
gradient demo == [('x',6),('x',6)]
gradient (Multiply 6 (Input 'x' 2) (Input 'y' 3)) == [('x',3),('y',2)]
gradient (Multiply 0 (Input 'x' 7) (Constant 0)) == [('x',0)]
gradient (Constant 7 :: Trace Char) == []
```

3 In this question, we test your knowledge of type inference.

a. [4pt] Consider the following data type that models a search that can fail, return a result, or branch into two searches:

```
data Search a = Dead | Found a | Split (Search a) (Search a)
```

Recall:

```
data Either a b = Left a | Right b
```

Give the most general type of expressions below, or write ill-typed. Each expression is worth 1 point. Prelude constructors and functions
have their usual types. Give types, not evaluated results.

```
Split (Found ['x']) Dead :: a. ..................................(1 pt.)

Found (Left Nothing) :: b. ......................................(1 pt.)

Found (map Just) :: c. ..........................................(1 pt.)

Split (Found []) (Found True) :: d. .............................(1 pt.)
```

b. [7pt] Give the most general type of each function below, or write "ill-typed". Use the type

```
concatMap :: (a -> [b]) -> [a] -> [b]
```

The functions to analyse are:

```
rerun f x = f x (f x)
rerun :: e. .....................................................(1 pt.)

collect f xs = concatMap (map f) xs
collect :: f. ...................................................(2 pt.)

agree f x y = f x == f y
agree :: g. .....................................................(2 pt.)

compareAt f x w = f (x + w) == f x
compareAt :: h. .................................................(2 pt.)
```

c. [6pt] Give a full type inference derivation for the following expression or show that it is ill-typed:

```
on replicate
```

Use the following signatures (on is imported from Data.Function):

```
replicate :: Int -> a -> [a]
on :: (b -> b -> c) -> (a -> b) -> a -> a -> c
```

6 pt. **i.**

4 In this question, we ask you to evaluate some simple Haskell expressions.

For each equality below, choose Correct, Incorrect or Don't know.

A correct judgement earns 1 point, an incorrect judgement loses 1 point, and Don't know or a blank answer earns 0.

For reference:

```
foldr f e [] = e
foldr f e (x:xs) = f x (foldr f e xs)

scanl f z [] = [z]
scanl f z (x:xs) = z : scanl f (f z x) xs

zipWith _ [] _ = []
zipWith _ _ [] = []
zipWith f (x:xs) (y:ys) = f x y : zipWith f xs ys
```

1 pt. **a.** foldr (\(x,w) acc -> (x,2*w) : acc) [] [('a',1),('b',0)] == [('a',2)]

- **a.** Correct
- **b.** Incorrect
- **c.** Don't know

1 pt. **b.** scanl (*) 1 [2,0,3] == [1,2,0,3]

- **a.** Correct
- **b.** Incorrect
- **c.** Don't know

1 pt. **c.** foldr (.) id [(+1),(*2)] 3 == 7

- **a.** Correct
- **b.** Incorrect
- **c.** Don't know

1 pt. **d.** zipWith (\w (x,s) -> (x,w*s)) [2,3] [('x',4),('x',5),('y',6)] == [('x',8),('x',15),('y',6)]

- **a.** Correct
- **b.** Incorrect
- **c.** Don't know

1 pt. **e.** sum [u*v | (_,u) <- [(1,2),(2,1)], (_,v) <- [(10,3),(20,0)]] == 9

a. Correct

b. Incorrect

c. Don't know